

OTIMIZAÇÃO DE MODELOS DE DIAGNÓSTICO

Algoritmos genéticos + *LLM* aplicados ao diagnóstico médico.

EQUIPE

Lais • Pedro • Renan

ESCOPO

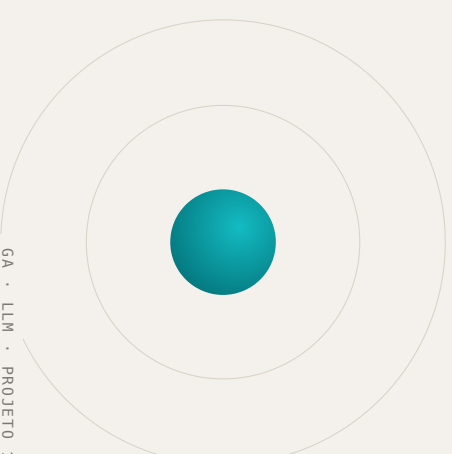
Projeto 1 • Módulo 1 como baseline

DURAÇÃO

~14 min • vídeo de demonstração

01 / 15

APRESENTAÇÃO



GA • LLM • PROJETO 1

ESCOPO ESCOLHIDO: PROJETO 1

Otimizar os modelos do Módulo 1 com **Algoritmo Genético** e integrar uma **LLM**.

- 01 Implementar um **Algoritmo Genético** do zero para otimizar hiperparâmetros — codificação de genes, seleção, cruzamento, mutação e função fitness baseada em métricas.
- 02 Monitoramento e logging dos experimentos, com escalabilidade em nuvem como item **opcional**.
- 03 Integrar uma **LLM pré-treinada** para gerar explicações em linguagem natural dos diagnósticos, com prompt engineering e avaliação de qualidade.

O fio condutor continua o mesmo da Fase 1: em diagnóstico, o falso negativo é o erro caro. A gente não pode deixar passar um caso maligno.

PROJETO 2 (ROTAS) NÃO FOI ESCOLHIDO

TRÊS BLOCOS, UM PACOTE PYTHON VERSIONADO

Baseline → Algoritmo Genético → LLM.

`src/models`

Bloco 1 · Baseline

Os cinco modelos do Módulo 1 e o dataset, portados para código versionado e testável.

`src/ga`

Bloco 2 · Algoritmo Genético

Recebe o espaço de busca de hiperparâmetros, evolui uma população e devolve o modelo otimizado com métricas.

`src/llm`

Bloco 3 · Camada de LLM

Recebe modelo otimizado + baseline e gera explicações em linguagem natural via API da OpenAI.

Rodamos tudo num ambiente isolado com Pipenv, com `src` separado de `tests` e `docs`. O baseline e o modelo otimizado são avaliados no **mesmo conjunto de teste** e mesmo `random_state` — assim a comparação é justa e o ganho reportado é confiável.

HERANÇA DA FASE 1

O número que o GA precisa bater.

Em `src/models/baseline.py` estão os cinco classificadores do Módulo 1 com hiperparâmetros escolhidos **na mão** — incluindo `class_weight 1` para 5.

Detalhe importante pra mais adiante: o baseline **não padronizava as *features*** e os hiperparâmetros foram escolhidos no olho, sem muito critério.

97,37%
Acurácia — Regressão Logística, melhor modelo do Módulo 1

97,67%
Recall da classe maligna — 1 falso negativo em 43 casos

0 CORAÇÃO DA FASE 2 – QUATRO ARQUIVOS, QUATRO EXIGÊNCIAS

Codificação · Operadores · Fitness · Loop.

encoding.py

Codificação

Cromossomo de reais entre 0–1. GeneSpec traduz para o hiperparâmetro real: log_float (C, 0.001–100), int (peso da classe), categorical (penalty).

operators.py

Operadores

Seleção por **torneio** (3 indivíduos). Cruzamento **uniforme**, taxa 0.8. Mutação **gaussiana** com clip nos limites.

fitness.py

Fitness

Pipeline **StandardScaler** + **classificador**. **Cross-validation estratificada 5 folds**. Score = 50% recall + 30% acurácia + 20% F1.

genetic_algorithm.py

Loop

Inicializa, avalia, aplica **elitismo** (top 2 passam direto) e repete por N gerações. random_state fixo + logging por geração.

- DEMO 1 · ALGORITMO GENÉTICO

O sistema rodando ao vivo.

```
$ python -m fase_2.tech_challenge.src.ga.experiments
# geração 01/30 | best=0.9312 avg=0.8940 | params=...
# geração 04/30 | best=0.9703 avg=0.9410 | params=...
# ...
```

Fizemos os três experimentos que o enunciado pede, cada um com um gráfico ao vivo: melhor score da geração (linha azul) e média da população (linha laranja). O resultado final fica gravado em `ga_results.json`.

`src/ga/experiments.py`

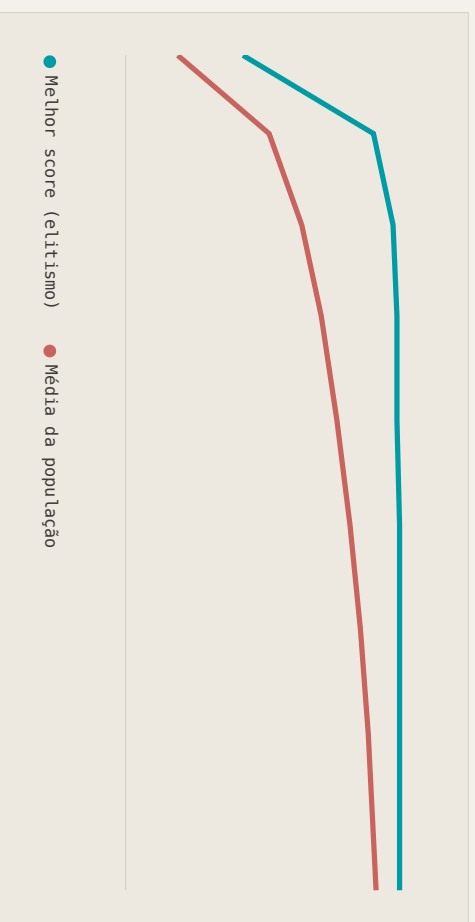
3 EXPERIMENTOS · POPULAÇÃO E MUTAÇÃO VARIADAS · 30 GERAÇÕES

O modelo otimizado supera o original.

MODELO	ACURÁCIA	RECALL	PRECISÃO	F1
LogReg original Módulo 1 · class_weight=5	0,9737	0,9767	0,9545	0,9655
LogReg otimizado (GA) C=0,35 · L2 · class_weight=2	0,9825	0,9767	0,9767	0,9767

Acurácia 97,37 → **98,25%** · precisão 95,45 → **97,67%** · F1 96,55 → **97,67%** — recall mantido em 0,9767 (1 FN em 43 malignos). O GA melhorou tudo sem sacrificar a métrica que mais importa.

O que mais chamou atenção: o GA encontrou `class_weight=2`, não 5 — a gente tinha exagerado ao escolher esse peso na mão no Módulo 1, e isso derrubava a precisão sem necessidade. O SVC Linear otimizado ficou em 96,49% — bom, mas não chegou a bater a Logistic Regression.



GA_PROGRESS_LOGISTIC_REGRESSION.PNG

Satura na geração 4, não é sorte.

O melhor score satura em **0,9703** já na geração 4 e se mantém estável — efeito do elitismo. Mas olha a média da população: sai de 0,916 e sobe até ~0,966. Ou seja, não é só um indivíduo sortudo, a **população inteira** vai melhorando.

Como fixamos o `random_state`, a execução é reproduzível. E os três experimentos chegam no mesmo campeão, o que dá confiança de que é realmente um bom ponto do espaço de busca — não coincidência.

A PARTE MAIS HONESTA DO TRABALHO

Na primeira versão, o GA **não batia** o baseline.

01 O fitness comparava cada indivíduo usando um **único split de validação**. Com uma base pequena (569 amostras), isso deixa o resultado com bastante variância — o campeão daquele split específico não generalizava bem.

02 A solução foi trocar por **validação cruzada estratificada de 5 folds** e incluir **StandardScaler** no pipeline. Foi essa mudança que levou o GA de 93% para 98% de acurácia.

03 E o interessante é que essas duas mudanças eram exatamente o que a gente tinha deixado como próximo passo no fechamento da Fase 1 — acabou fechando esse ciclo.

93% → 98% DE ACURÁCIA

API DA OPENAI · TEMPERATURA 0,3 · CHAVE EM .ENV

A LLM **redige**, nunca decide.

`explain.py` – `grounding`

O cálculo é matemático, não da LLM

Como os modelos otimizados são lineares dentro de um pipeline, o próprio código já calcula qual feature pesou mais na predição: coeficiente vezes valor padronizado da amostra. A LLM recebe essa lista pronta e só **redige** em linguagem natural — assim ela não tem chance de inventar um motivo clínico que o modelo não usou de fato.

`prompts.py` – `prompt engineering`

Dois templates, regras rígidas

Um explica um **diagnóstico individual**, outro o **comparativo** GA vs. baseline. Regras do prompt de sistema: usar só os números fornecidos, nunca certeza absoluta, sem jargão de ML, estrutura em 3 partes (resultado, fatores, recomendação), sempre reforçando que a decisão final é do profissional de saúde.

• DEMO 2 · LLM PONTA A PONTA

De número para linguagem clínica.

```
$ python -m fase_2.tech_challenge.smoke_test_llm
# 1. treina Regressão Logística no dataset real
# 2. explica o diagnóstico de UMA paciente do teste
# 3. explica o comparativo GA vs. baseline
```

No passo 2, a LLM devolve a confiança do modelo, os fatores que pesaram na decisão e uma recomendação prática — sem jargão, sempre deixando claro que quem decide é o médico. No passo 3, ela traduz algo como "a precisão subiu" para "o modelo passou a errar menos ao classificar casos malignos como benignos" — pensado pra quem não é técnico.

smoke_test_llm.py

TEST_GA.PY

Decodificação nos limites, operadores, fitness.

Teste ponta a ponta verifica uma propriedade forte: **com elitismo, o melhor score nunca cai entre gerações.**

TEST_LLM.PY

Grounding garantido, sem custo de API.

Números sempre vêm do código, nunca da LLM. Chamada **mockada** — suíte rápida, determinística, sem custo.

```
$ python -m pytest fase_2/tech_challenge/tests/ -v
```

TRANSPARÊNCIA SOBRE O ITEM 2 DO ENUNCIADO

Logging implementado. Nuvem, próximo passo.

LOGGING · IMPLEMENTADO

AUTOSCALE AZURE · OPCIONAL / PENDENTE

O GA registra cada geração durante a execução. Já a escalabilidade automática em nuvem (Azure) o próprio enunciado marca como **opcional, pra pontuação extra** — ficou como próximo passo mesmo. A dependência do Azure Monitor já está no projeto e a arquitetura já está pensada pra receber isso; falta plugar o deploy.

ARQUITETURA PREPARADA PARA DEPLOY

TAKEAWAYS DA FASE 2

O Algoritmo Genético não só otimizou — **corrigiu uma decisão de domínio** tomada por intuição.

- A gente tinha escolhido **class_weight** 5 na mão, meio no chute — o GA achou que 2 funciona melhor, e venceu em todas as métricas.
- A gente já tinha apontado isso no fechamento da Fase 1: faltava **validação cruzada e padronizar as features**. Foi exatamente isso que fez o GA funcionar de verdade.
- A LLM entrou como **tradutora, não oráculo**: ela nunca decide o diagnóstico nem escolhe as features.

*A gente otimiza,
explica e prioriza —
não substitui.*

MÉDICO MANTÉM A DECISÃO CLÍNICA